

Second exercise in ML: a regression algorithm

Goal: Implement a **regression** using a Deep Neural Network in Keras. You need to invent a function of five variables (x_1 , x_2 , x_3 , x_4 , x_5), generate some data, and create a feed forward model (with one or more hidden layers).

When building the model, consider what should change compared to the classification problem: what is the appropriate loss function to use? Which activation function should be placed in the last layer? Finally, try to make a histogram of the residuals on the validation sample, where `residuals = (y_predicted - y_truth)`.

```
In [ ]: import numpy as np
import matplotlib.pyplot as plt
from matplotlib.colors import ListedColormap
from keras.layers import Input,Dense,Dropout,Flatten,LeakyReLU
from keras.models import Model
```

Generate some data

```
In [ ]: from math import *
# Fixing random state for reproducibility
np.random.seed(123)

# Try changing this
theFunction = lambda x1,x2,x3,x4,x5 : (np.sin(x1*10)+x2**x3)*x4

N=800
x1 = 1.0 * np.random.rand(N)
x2 = 1.0 * np.random.rand(N)
x3 = 1.0 * np.random.rand(N)
x4 = 1.0 * np.random.rand(N)
x5 = 1.0 * np.random.rand(N)
y=theFunction(x1,x2,x3,x4,x5)

X = np.stack((x1,x2,x3,x4,x5), axis=-1)

print("X[:10,:] = ", X[:10,:])
print("y[:10] = ", y[:10])

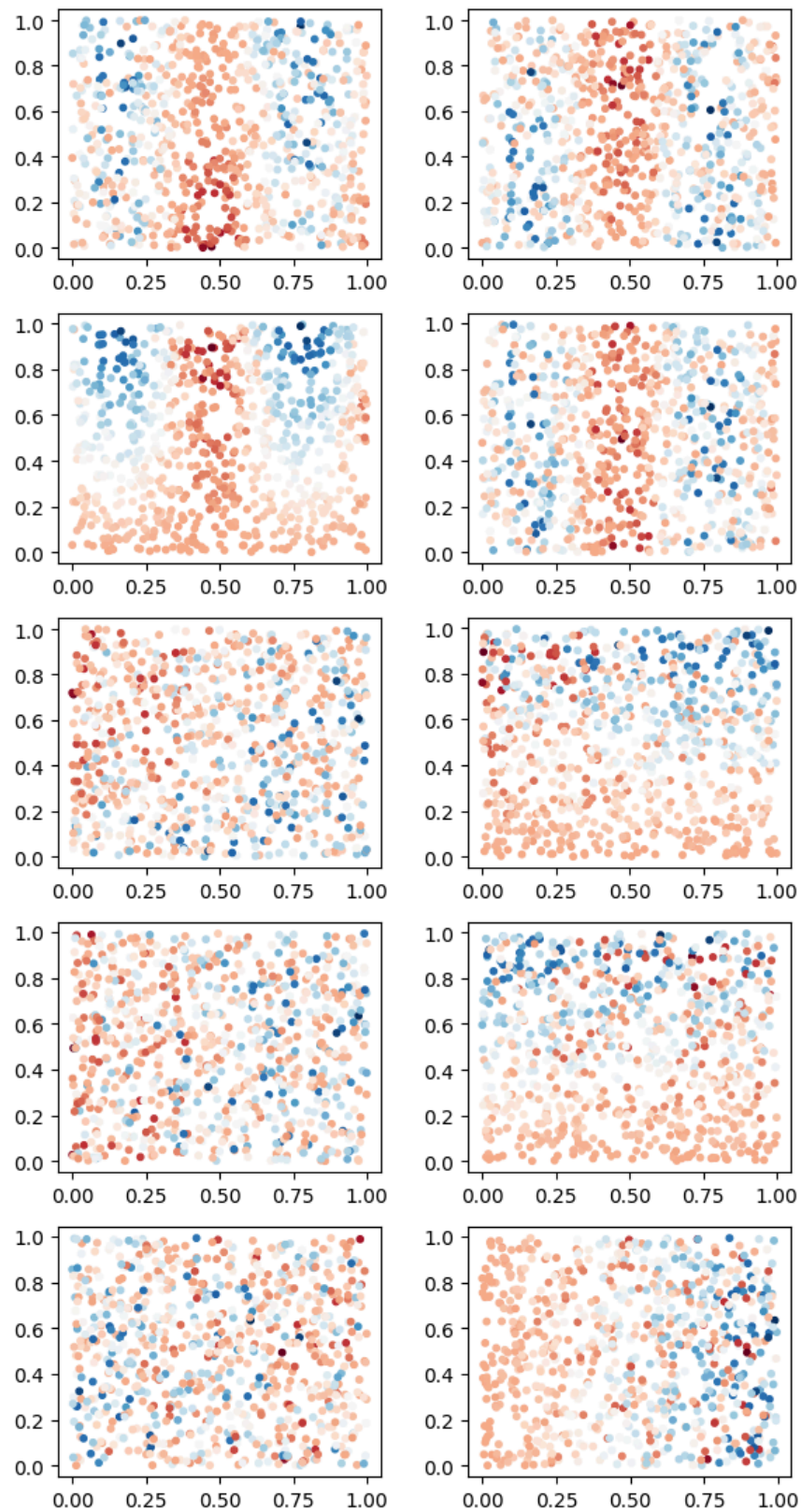
X[:10,:] = [[0.69646919 0.3871952 0.4732499 0.65353053 0.29974806]
[0.28613933 0.55585958 0.12527156 0.97839806 0.75045588]
[0.22685145 0.01414381 0.07528317 0.32387778 0.29155519]
[0.55131477 0.847647 0.66483846 0.13137077 0.51634825]
[0.71946897 0.92191986 0.83790816 0.17253149 0.48778668]
[0.42310646 0.55052969 0.92409405 0.03216338 0.04743733]
[0.9807642 0.26802112 0.38818585 0.82387829 0.57038305]
[0.68482974 0.990239 0.93831583 0.07615528 0.5049035 ]
[0.4809319 0.38319403 0.81980347 0.89366009 0.51916279]
[0.39211752 0.69365539 0.42404167 0.04843728 0.0318487 ]]
y[:10] = [ 0.8288136 1.17957967 0.48323524 0.02624378 0.2975435 -0.00998148
0.18639801 0.11623952 -0.48240599 0.00742746]
```

Do some *data exploration* by drawing some of the data points.

```
In [ ]: fig, axes = plt.subplots(nrows=5, ncols=2, figsize=(6, 11))
axes = axes.flatten()

idx = 0
for i in range(5):
    for j in range(i+1, 5):
        ax = axes[idx]
        ax.scatter(X[:, i], X[:, j], c=y, s=10, cmap=plt.cm.RdBu#, edgecolors='k')
        idx += 1

plt.tight_layout()
plt.show()
```



Create your MLP with the right activation and loss function.

```
In [ ]: inputs = Input(shape=(5,))
hidden = Dense(500, activation='relu')(inputs)
outputs = Dense(1)(hidden)
model = Model(inputs=inputs, outputs=outputs)
model.compile(loss='MSE', optimizer='adam')
model.summary()
```

Model: "functional"

Layer (type)	Output Shape	Param #
input_layer (InputLayer)	(None, 5)	0
dense (Dense)	(None, 500)	3,000
dense_1 (Dense)	(None, 1)	501

Total params: 3,501 (13.68 KB)

Trainable params: 3,501 (13.68 KB)

Non-trainable params: 0 (0.00 B)

Split the dataset in training+validation and test, then fit.

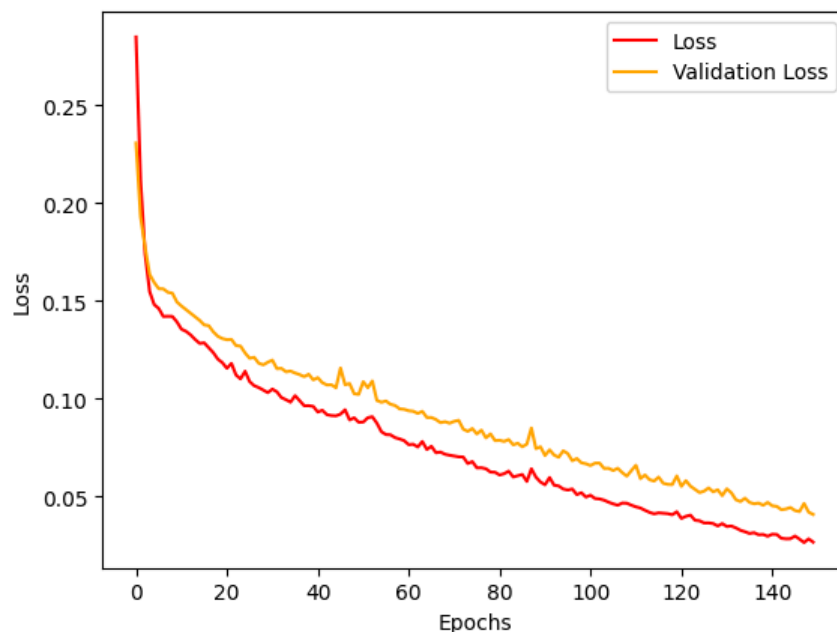
Note: This time we did the splitting "by hand". You can also use the `train_test_split()` function from `sklearn.model_selection` as shown in the comments below.

```
In [ ]: #from sklearn.model_selection import train_test_split
#X_tr, X_val, y_tr, y_val = train_test_split(X,y,test_size=0.5)

nsplit = int(N/2)
X_tr = X[:nsplit,:]
X_val = X[nsplit:,:]
y_tr = y[:nsplit]
y_val = y[nsplit:]

# You can check if the splitting makes sense by uncommenting these lines
#print(X_tr[:10])
#print(y_tr[:10])
#print(X_val[:10])
#print(y_val[:10])

train_output = model.fit(X_tr, y_tr, validation_data=(X_val,y_val), epochs=150, verbose=0)
plt.plot(train_output.history["loss"], color="red", label="Loss")
plt.plot(train_output.history["val_loss"], color="orange", label="Validation Loss")
plt.legend()
plt.xlabel("Epochs")
plt.ylabel("Loss")
plt.show()
```



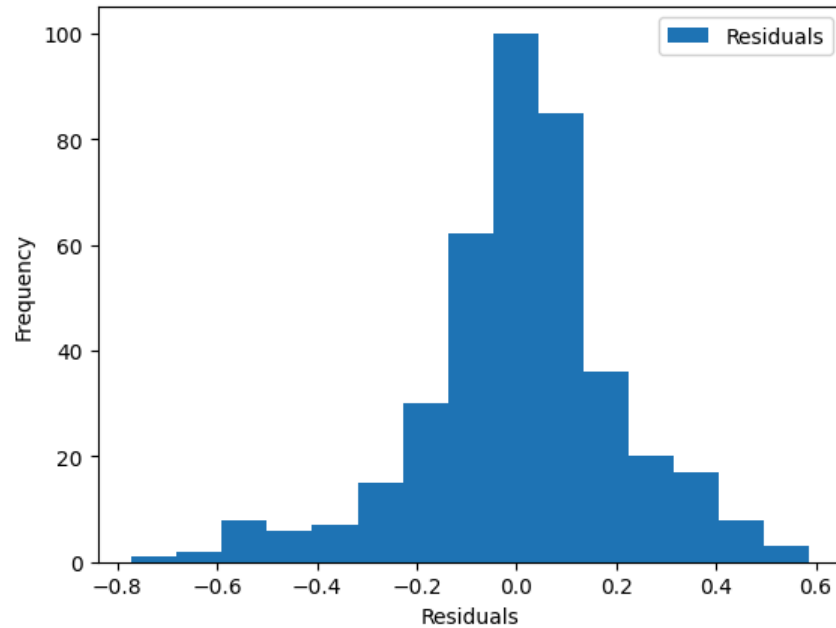
Predict the results on your validation dataset and compute the residuals of the regression. Mind the fact that the shape of `y_val` and `y_pred_val` are different.

```
In [ ]: y_pred_val = model.predict(X_val)

# Mind the shape of the prediction and how we fixed the problem
print(y_val.shape, y_pred_val.shape, y_pred_val[:,0].shape)
residuals = np.subtract(y_pred_val[:,0], y_val)
plt.hist(residuals, 15, label="Residuals")
plt.legend()
plt.xlabel("Residuals")
```

```
plt.ylabel("Frequency")
plt.show()
```

13/13 — 0s 13ms/step
(400,) (400, 1) (400,)



Repeat the computations using a DNN.

```
In [ ]: inputs = Input(shape=(5,))
hidden = Dense(15, activation='relu')(inputs)
hidden = Dense(15, activation='relu')(hidden)
hidden = Dense(15, activation='relu')(hidden)
hidden = Dense(15, activation='relu')(hidden)
hidden = Dense(15, activation='relu')(hidden)

hidden = Dense(15, activation='relu')(hidden)
outputs = Dense(1)(hidden)

deepmodel = Model(inputs=inputs, outputs=outputs)
deepmodel.compile(loss='MSE', optimizer='adam')
deepmodel.summary()
```

Model: "functional_1"

Layer (type)	Output Shape	Param #
input_layer_1 (InputLayer)	(None, 5)	0
dense_2 (Dense)	(None, 15)	90
dense_3 (Dense)	(None, 15)	240
dense_4 (Dense)	(None, 15)	240
dense_5 (Dense)	(None, 15)	240
dense_6 (Dense)	(None, 15)	240
dense_7 (Dense)	(None, 15)	240
dense_8 (Dense)	(None, 1)	16

Total params: 1,306 (5.10 KB)

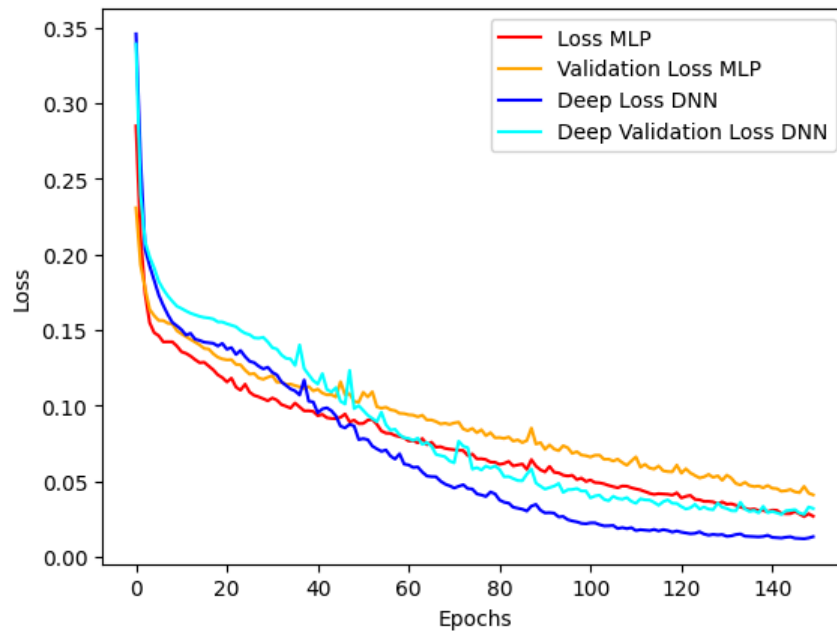
Trainable params: 1,306 (5.10 KB)

Non-trainable params: 0 (0.00 B)

```
In [ ]: deep_train_output = deepmodel.fit(X, y, validation_split=0.5, epochs=150, verbose=0)

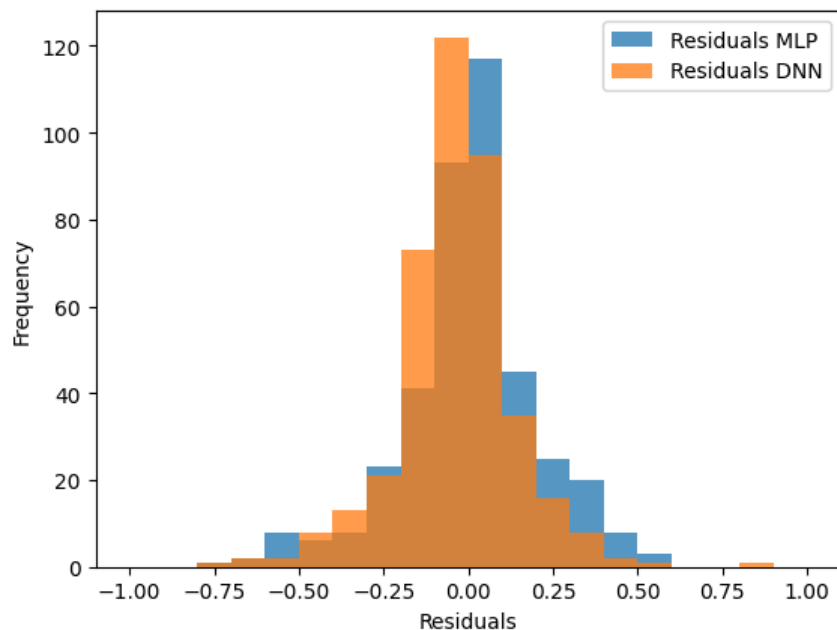
plt.plot(train_output.history["loss"], color="red", label="Loss MLP")
plt.plot(train_output.history["val_loss"], color="orange", label="Validation Loss MLP")
plt.plot(deep_train_output.history["loss"], color="blue", label="Deep Loss DNN")
plt.plot(deep_train_output.history["val_loss"], color="cyan", label="Deep Validation Loss DNN")
plt.legend()
plt.xlabel("Epochs")
```

```
plt.ylabel("Loss")
plt.show()
```



```
In [ ]: y_pred_val_deep = deepmodel.predict(X_val)
residuals_deep = np.subtract(y_pred_val_deep[:,0],y_val)
plt.hist(residuals, 20, range=[-1,1], alpha=0.75, label="Residuals MLP")
plt.hist(residuals_deep, 20, range=[-1,1], alpha=0.75, label="Residuals DNN")
plt.legend()
plt.xlabel("Residuals")
plt.ylabel("Frequency")
plt.show()
```

13/13 ————— 1s 18ms/step



Try and make your own considerations about the performance of the two models.

Note: Parameter Count and Dataset Size

As a general rule of thumb, the number of trainable parameters in a model should roughly match the order of magnitude of the available training data. This principle is analogous to solving a system of equations: determining a given number of unknown variables requires an equivalent number of constraints.

If a model is significantly over-parameterized (such as using 3500 parameters for a dataset of 800 samples) the problem becomes under-constrained. This creates a vast space of equivalent solutions, making it difficult for the network to converge on an optimal one. Consequently, an overly complex deep network can often result in worse overall performance compared to a shallower, appropriately constrained model.